TP – Stockage Temporaire des Données dans Android

BTS DAI – Développement Mobile

Objectifs du TP

À la fin de ce TP, l'étudiant sera capable de :
— Comprendre la différence entre stockage persistant et stockage temporaire
— Utiliser la mémoire volatile (RAM)
— Mettre en place un système de cache
— Gérer une session utilisateur dans une application Android
— Afficher des données temporaires dans une interface utilisant **StackView**

Introduction

Dans une application mobile, certaines données ne doivent pas forcément être sauvegardées de manière permanente dans une base de données.

On parle alors de **stockage temporaire**.

Ce stockage permet :
— d'améliorer les performances
— de réduire les accès réseau
— d'accélérer l'affichage des données

Dans ce TP nous allons étudier trois mécanismes :
— le **Cache**
— la **Mémoire volatile**
— la **Session utilisateur**

1 Partie 1 – Mémoire Volatile (RAM)

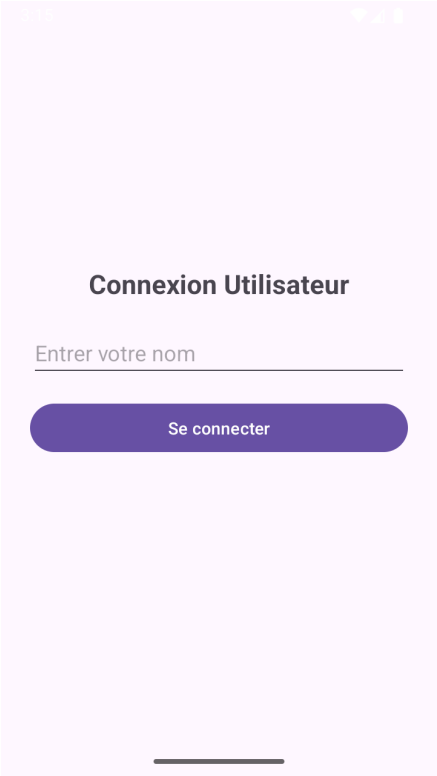
1.1 Principe

La mémoire volatile correspond aux variables stockées en RAM pendant l'exécution de l'application.

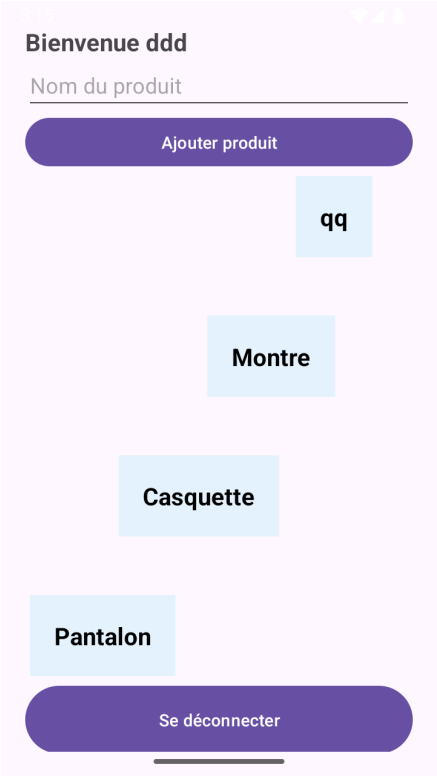
Caractéristiques :
— Très rapide
— Disponible uniquement pendant l'exécution
— Les données sont perdues lorsque l'application est fermée

1.2 Exemple Théorique

Une variable Java peut être utilisée pour stocker temporairement une information.



(a) Liste StackView contenant les données



(b) Interface Login

FIGURE 1 – Interfaces de l'application

```
public class SessionManager {

    public static String utilisateurConnecte;

}
```

Utilisation :

```
SessionManager.utilisateurConnecte = "Ahmed";
```

Lecture :

```
String user = SessionManager.utilisateurConnecte;
```

1.3 Travail demandé

Créer une application contenant :

— un champ EditText pour saisir le nom de l'utilisateur
— un bouton "Se connecter"
— un TextView affichant l'utilisateur connecté

Consignes

— créer une classe **SessionManager**
— stocker le nom de l'utilisateur dans une variable statique
— afficher l'utilisateur connecté dans l'activité principale

1.4 Partie pratique

Compléter la classe suivante :

```
public class SessionManager {

    private static ______ utilisateur;

    public static void setUtilisateur(String u){
        ______ = u;
    }

    public static String getUtilisateur(){
        return ______;
    }

}
```

2 Partie 2 – Le Cache

2.1 Principe

Le cache permet de stocker temporairement des données afin d'éviter de les recalculer ou de les télécharger plusieurs fois.

Exemples :

— images téléchargées depuis internet
— informations produits
— résultats d'une API

Android propose une classe appelée :

LruCache

Elle supprime automatiquement les données les moins utilisées.

2.2 Exemple Théorique

Création d'un cache :

```
int cacheSize = 4 * 1024 * 1024;

LruCache<String, String> cache =
        new LruCache<>(cacheSize);
```

Ajouter une donnée :

```
cache.put("produit1", "Chaussures␣Nike");
```

Lire une donnée :

```
String produit = cache.get("produit1");
```

2.3 Travail demandé

Créer un système de cache pour stocker des produits.
Chaque produit possède :
— un identifiant
— un nom
Étapes
— créer un cache avec LruCache
— ajouter 5 produits dans le cache
— récupérer un produit par son identifiant

```
public class CacheManager {

    public static ______<String,String> cache;

    public static void initCache(){
     if(cache == null){

        int cacheSize = 4 * 1024 * 1024;

        cache = new ______<>(cacheSize);

        cache.put("1","________");
        cache.put("2","________");
        cache.put("3","________");

    }}

}
```

3 Partie 3 : Connexion utilisateur

3.1 Partie théorique

Dans l'activité principale, nous récupérons le nom de l'utilisateur et nous l'enregistrons dans la session.
Exemple :

```
btnLogin.setOnClickListener(v -> {

    String nom = editNom.getText().toString();

    SessionManager.setUtilisateur(nom);

    Intent i = new Intent(MainActivity.this,
            ProduitsActivity.class);

    startActivity(i);
```

```
});
```

4 Partie 3 – Affichage des données avec StackView

Pour afficher les produits, nous allons utiliser un composant Android appelé : **StackView**
Il permet d'afficher les éléments comme une pile de cartes.

4.1 Exemple XML

```
<StackView
    android:id="@+id/stackView"
    android:layout_width="match_parent"
    android:layout_height="300dp"/>
```

4.2 Principe

Le StackView utilise un **Adapter** pour afficher les données.

4.3 Exemple Théorique

```
ArrayList<String> produits = new ArrayList<>();

produits.add("Produit␣1");
produits.add("Produit␣2");
produits.add("Produit␣3");

ArrayAdapter<String> adapter =
        new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1,
        produits);

stackView.setAdapter(adapter);
```

4.4 Travail demandé

Afficher les produits stockés dans le cache dans un StackView.
Consignes
- récupérer les produits du cache
- les stocker dans un ArrayList
- afficher la liste dans un StackView

Compléter le code suivant :

```
ArrayList<String> produits = new __________<>();
```

```
produits.add(CacheManager.cache.get("___"));
produits.add(CacheManager.cache.get("___"));
produits.add(CacheManager.cache.get("___"));
```

5 Partie 4 – Session Utilisateur

5.1 Principe

Une session permet de conserver les informations d'un utilisateur pendant l'utilisation de l'application.

Exemples :

— utilisateur connecté
— panier d'achat
— préférences temporaires

La session est généralement stockée :

— en RAM
— dans une classe singleton

5.2 Exemple Théorique

```
public class Session {

    private static String user;

    public static void setUser(String u){
        user = u;
    }

    public static String getUser(){
        return user;
    }

}
```

Utilisation :

```
Session.setUser("Sara");

String u = Session.getUser();
```

5.3 Travail demandé

Créer une session utilisateur dans l'application.

Fonctionnement :

— l'utilisateur saisit son nom
— l'application enregistre la session
— le nom de l'utilisateur apparaît sur toutes les activités

6 Partie 6 : Ajouter un produit

6.1 Partie théorique

L'utilisateur peut ajouter un produit dans le cache.

```
CacheManager.cache.put("6","Sac");
```

6.2 Partie pratique

Compléter le code suivant :

```
btnAjouter.setOnClickListener(v -> {

    String produit = editProduit.getText().toString();

    CacheManager.cache.______(String.valueOf(idProduit),
                             produit);

    produits.______(produit);

    adapter.________________();

});
```

7 Partie 6 : Déconnexion

7.1 Partie théorique

La déconnexion consiste à supprimer la session utilisateur et revenir à l'écran de connexion.

```
SessionManager.logout();

Intent intent = new Intent(
        ProduitsActivity.this,
        MainActivity.class);

startActivity(intent);

finish();
```

7.2 Partie pratique

Compléter le code suivant :

```
btnLogout.setOnClickListener(v -> {
```

```
    SessionManager.________();

    Intent intent = new Intent(
            ProduitsActivity.this,
            ____________.class);

    startActivity(intent);

    ______();

});
```

8 Exercice Bonus

Créer une mini application contenant :
- un écran de connexion
- un écran affichant des produits
- les produits sont stockés dans un cache
- l'utilisateur connecté est conservé dans une session
- les produits sont affichés dans un StackView

9 Questions de réflexion

1. Quelle est la différence entre mémoire volatile et stockage SQLite ?
2. Pourquoi utiliser un cache dans une application mobile ?
3. Que se passe-t-il si l'application est fermée lorsque les données sont uniquement en RAM ?
4. Dans quel cas utiliser une session utilisateur ?